

Collecting Beepers

Day 3 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“each of the beepers, and return back again”*
- ▶ *“never be more than \$10\$ beepers”*

Programming interpretation:

- ▶ The grid size is irrelevant once coordinates are known.
- ▶ With at most 10 beepers, bitmask dynamic programming over visited sets is feasible.
- ▶ The state is current point plus the set of collected beepers.

Textbook connection: CP4 3.5 e. Travelling Salesman Problem

Core technique: Bitmask DP for small TSP

- ▶ Start at the robot's home position.
- ▶ Visit every beeper exactly once in any order.
- ▶ Return to the starting position with minimum Manhattan distance.

- ▶ The number of beepers is small, but all visit orders are factorial.
- ▶ A bitmask compactly records which beepers are already collected.
- ▶ The current location plus mask fully determines the remaining problem.

- ▶ Store `points[0]` as the start and `points[1..B]` as beepers.
- ▶ Define `dp(current, mask)` as minimum remaining route length.
- ▶ If mask has all `B` bits set, return distance back to the start.
- ▶ Otherwise try every unvisited beeper, pay travel distance, and recurse.
- ▶ Memoize `dp` with `lru_cache`.

- ▶ Bit i of mask corresponds to beeper i at points $[i + 1]$.
- ▶ $\text{new_mask} = \text{mask} \mid (1 \ll i)$ marks a beeper as visited.
- ▶ Manhattan distance is $\text{abs}(dx) + \text{abs}(dy)$.

- ▶ Every feasible tour has a first unvisited beeper chosen from the current state.
- ▶ The recurrence tries every such next beeper and takes the minimum.
- ▶ Memoization does not change the recurrence; it only avoids recomputation.

Complexity: $O(B^2 * 2^B)$ time and $O(B * 2^B)$ memory.

- ▶ The grid dimensions are read but not needed for distance calculations.
- ▶ This is the classic small TSP state: position plus visited set.

Source lines: beepers.py:45-52

```
import sys
from functools import lru_cache

def distance(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

What this chunk does

- ▶ Karel moves only horizontally and vertically.
- ▶ Manhattan distance is therefore the exact travel cost between two coordinates.

Source lines: beepers.py:54-76

```
def solve_case():  
    width, height = map(int, input().split())  
  
    start_x, start_y = map(int, input().split())  
  
    num_beepers = int(input())  
  
    points = [(start_x, start_y)]  
  
    for _ in range(num_beepers):  
        x, y = map(int, input().split())  
        points.append((x, y))  
  
    all_visited = (1 << num_beepers) - 1
```

What this chunk does

- ▶ `points[0]` is the starting location; later entries are beepers.
- ▶ The `all_visited` mask has one bit set for each beeper.
- ▶ When the current mask equals `all_visited`, collection is complete.

Source lines: beepers.py:78-95

```
@lru_cache(None)
def dp(current, mask):

    if mask == all_visited:
        return distance(points[current], points[0])
```

What this chunk does

- ▶ lru_cache stores answers for repeated current/mask states.
- ▶ The base case returns directly to the starting point.
- ▶ This turns the factorial route search into a bitmask DP.

Source lines: beepers.py:97-124

```
best = float("inf")

for beeper in range(num_beeper):

    if mask & (1 << beeper):
        continue

    next_point = beeper + 1

    new_mask = mask | (1 << beeper)

    travel = distance(points[current], points[next_point])

    total = travel + dp(next_point, new_mask)

    best = min(best, total)

return best

return dp(0, 0)
```

What this chunk does

- ▶ The loop skips any beeper whose bit is already set.
- ▶ new_mask marks the chosen beeper as collected.
- ▶ The best recursive choice is the minimum travel plus future cost.

Source lines: beepers.py:127-138

```
def main():  
    global input  
    input = sys.stdin.buffer.readline  
  
    test_cases = int(input())  
  
    for _ in range(test_cases):  
        answer = solve_case()  
        print(answer)  
  
main()
```

What this chunk does

- ▶ The main loop processes every Karel scenario independently.
- ▶ solve_case returns the minimum round-trip distance.

Solution file: `practice_contests/day_3/beepers.py`

Problem: <https://open.kattis.com/problems/beepers>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.